

An AI-powered Global Career Intelligence Platform that automatically discovers, verifies, categorizes, analyzes, and serves worldwide job opportunities from company career pages with real-time updates, intelligent search, and personalized recommendations.

This demonstrates:

- Distributed systems
- Web automation
- AI integration
- Search engines
- Queue systems
- Background workers
- Production deployment
- Monitoring
- Caching
- System Design

These are exactly the kinds of engineering topics interviewers at large companies ask about.

Features

1. Automated Career Crawlers

Instead of manually adding every job,

Admin only adds:

<https://careers.company.com>

Crawler automatically

- ✓ discovers jobs
- ✓ extracts details
- ✓ categorizes

✓ updates database

2. AI Job Parser

Most descriptions are huge.

AI converts

50 lines

↓

Role Summary

Required Skills

Preferred Skills

Experience

Responsibilities

Tech Stack

Benefits

3. Skill Graph

Every skill becomes searchable.

Example

Python



Companies hiring



Google

Microsoft

Tesla

Amazon

Stripe

4. Duplicate Detection

Same job appears on

Company Site

LinkedIn

Indeed

Remove duplicates.

5. Link Health Monitoring

Every 24 hours

Check

200 OK

↓

Keep

404

↓

Delete

Redirect

↓

Update

Blocked

↓

Retry

6. Salary Intelligence

AI estimates salaries when not listed.

Company

Country

Experience

↓

Estimated Salary

7. Trending Skills

Example

Last 30 Days

Top Skills

Python ↑

Rust ↑

GenAI ↑

LangGraph ↑

Power BI ↓

8. Hiring Heat Map

Interactive world map.

USA

12,450 jobs

India

18,000

Germany

7,300

Japan

2,500

9. AI Career Advisor

User says

I know

Python

SQL

React

AI answers

Backend Developer

Data Analyst

Full Stack

AI Engineer

10. Resume Matching

Upload Resume

↓

AI extracts skills

↓

Matches against jobs

↓

Shows

92%

Google

87%

Amazon

95%

Stripe

11. Interview Resources

Every role contains

Projects

Interview Questions

Roadmaps

Courses

Books

Certifications

12. Company Insights

Every company page

Contains

Hiring Trends

Tech Stack

Open Roles

Hiring Locations

Average Salary

Remote %

Reviews Link

13. Job Timeline

Track

Posted

Updated

Expired

Closed

14. Email Alerts

User subscribes

Backend

Germany

New job appears

↓

Email sent

15. Telegram Bot

/python jobs india

↓

Returns

Latest Jobs

16. Discord Bot

Same.

17. REST API

Expose

GET

/jobs

/companies

/skills

/roles

/countries

18. GraphQL API

Optional.

19. Admin Analytics

Dashboard

Jobs Crawled Today

Broken Links

Companies Online

Crawler Health

Average Crawl Time

Queue Size

20. System Health Dashboard

Monitor

CPU

Memory

Redis

Mongo

Workers

Queue

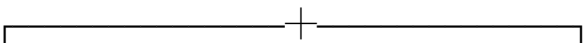
Errors

Production Architecture

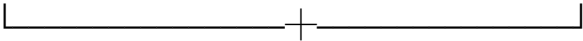
Next.js Frontend



API Gateway (Express)



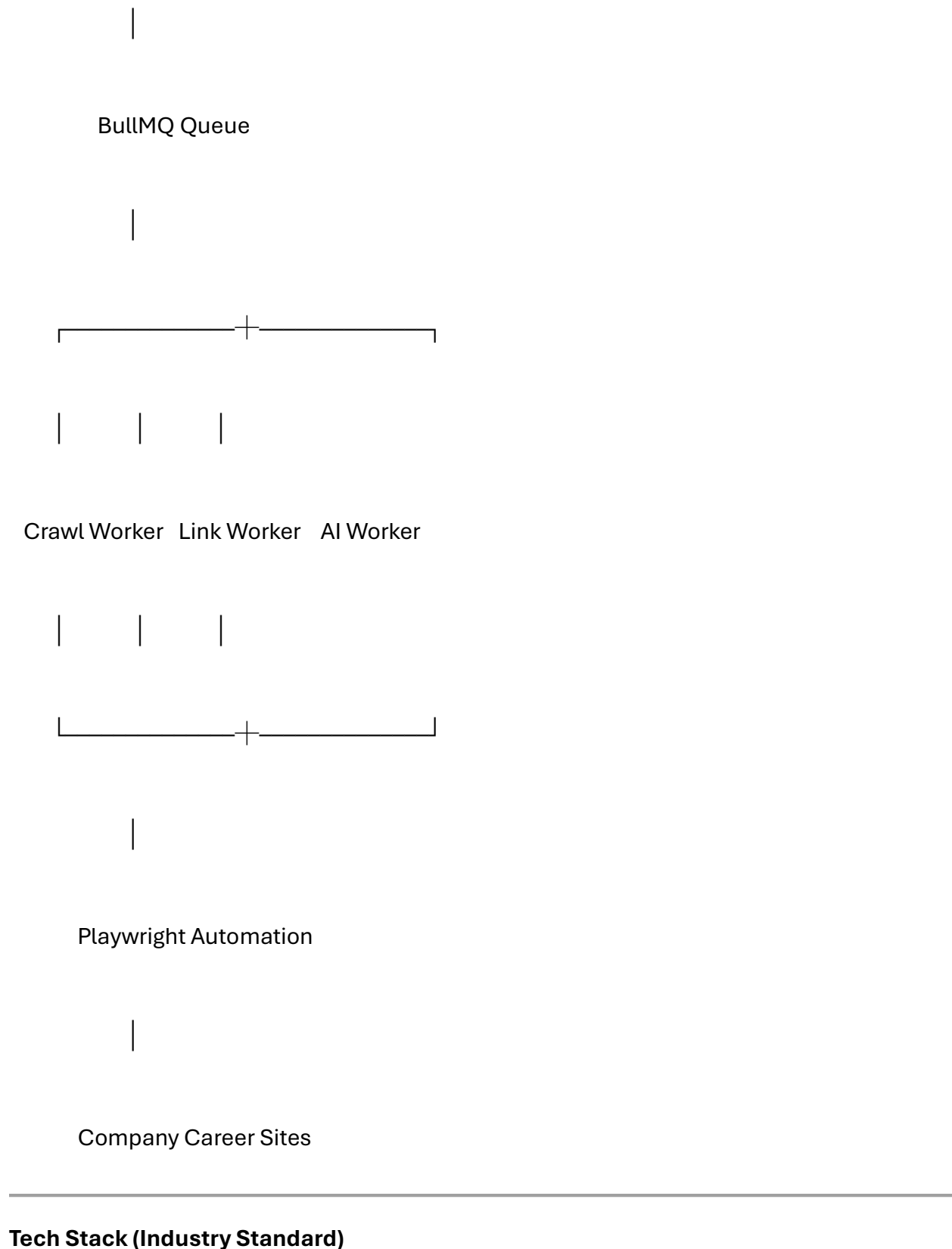
Auth Service Search Service Admin Service



MongoDB Cluster



Redis Cache Cluster



Frontend

✓ Next.js

React

TypeScript

TailwindCSS

Shadcn UI

TanStack Query

Zustand

React Hook Form

Zod

Framer Motion

Backend

Node.js

Express

TypeScript

Prisma ORM

MongoDB

Redis

BullMQ

JWT

Google OAuth

Swagger

Automation

Playwright

Cheerio

Puppeteer (fallback)

XPath selectors

CSS selectors

Browser contexts

AI

Google Gemini

LangChain

Embeddings (optional)

Search

Meilisearch (free and lightweight)

or

Elasticsearch

Queue

BullMQ

Redis

Storage

MongoDB Atlas Free

Cloudinary

Authentication

Google OAuth

GitHub OAuth

JWT

Refresh Tokens

RBAC

Monitoring

Prometheus

Grafana

Sentry

Morgan

Winston

DevOps

Docker

Docker Compose

GitHub Actions

Nginx

Ubuntu VPS

Deployment

Frontend

Vercel

Backend

Railway

Render

or

Oracle Cloud Free Tier

Free Services

MongoDB Atlas

Redis Cloud Free

Cloudinary Free

GitHub Actions

Vercel

Render

Railway

Google Gemini API

Resend (emails)

GitHub Student Pack (if eligible)

Folder Structure

career-platform

apps/

frontend

backend

worker-crawler

worker-verifier

worker-ai

packages/

ui

shared

database

auth

crawler

search

email

notification

docker/

monitoring/

nginx/

scripts/

docs/

Database Collections

Users

Companies

Jobs

Skills

Countries

Roles

Applications

Notifications

CrawlerLogs

WorkerLogs

SearchIndex

CompanyRules

ExpiredJobs

ResumeMatches

Analytics

What to Build First (Roadmap)

Phase 1 – Foundation

- Monorepo setup (TurboRepo or Nx)
- Next.js frontend
- Express backend
- MongoDB connection
- Authentication (JWT + Google OAuth)
- Role-based access (Admin/User)

Phase 2 – Core Job Platform

- Companies CRUD
- Roles CRUD
- Countries CRUD
- Jobs CRUD
- Search & filters
- Responsive UI

Phase 3 – Automation Engine

- Playwright crawler
- Company-specific adapters
- Scheduled crawling
- Queue with BullMQ
- Job extraction
- Deduplication

Phase 4 – Link Verification

- HTTP health checks
- Retry logic
- Expired job detection
- Automatic cleanup
- Crawl logs

Phase 5 – AI Features

- Job description summarization
- Skill extraction
- AI role recommendations
- Resume-to-job matching
- Career advisor chatbot

Phase 6 – Search & Discovery

- Full-text search
- Skill filters
- Country filters
- Salary filters
- Remote/Hybrid/On-site filters

Phase 7 – Notifications

- Email alerts

- Push notifications
- Telegram/Discord integration (optional)
- Saved searches

Phase 8 – Analytics & Monitoring

- Admin dashboard
- Queue monitoring
- Worker health
- Metrics (Prometheus/Grafana)
- Error tracking (Sentry)

Phase 9 – Deployment

- Dockerize all services
- CI/CD with GitHub Actions
- Production database
- Reverse proxy (Nginx)
- HTTPS
- Monitoring and backups

Important Design Decision

Don't create **one universal crawler**. Every careers site is structured differently, and many use different frameworks or APIs.

Instead, build a **Crawler Adapter System**:

Crawler Core

|

|— Google Adapter

|— Microsoft Adapter

|— Amazon Adapter

- |— Apple Adapter
- |— Meta Adapter
- |— NVIDIA Adapter
- |— Oracle Adapter
- |— Infosys Adapter
- |— TCS Adapter
- └— Generic Adapter

Each adapter knows how to navigate and extract jobs for its target site, while the core engine handles scheduling, retries, logging, and persistence. This pattern is much more maintainable and mirrors how production data-ingestion systems are often designed.
